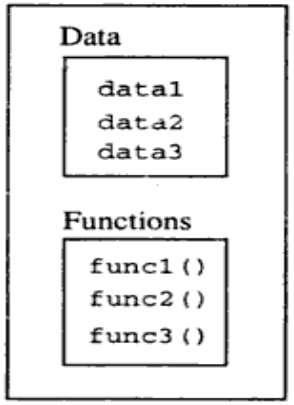
Chapter – 5

Classes and Objects

Classes & Objects



Class grouping data and functions

Object-oriented programming constructs modeled out of data types called *classes*. Defining variables of a class data type is known as a *class instantiation* and such variables are called *objects*. (Object is an instance of a class.) A class encloses both the *data* and *functions* that operate on the data, into a *single unit* The variables and functions enclosed in a class are called *data members* and *member functions* respectively. Member functions define the permissible operations on the data members of a class.

Using Class in C++ needs 3 steps to be followed

1. Specify the class

   i. Declaration of class
   ii. Defintion of member functions

2. Create objects for the class

3. Access the public members of the class using objects

Specifying a class

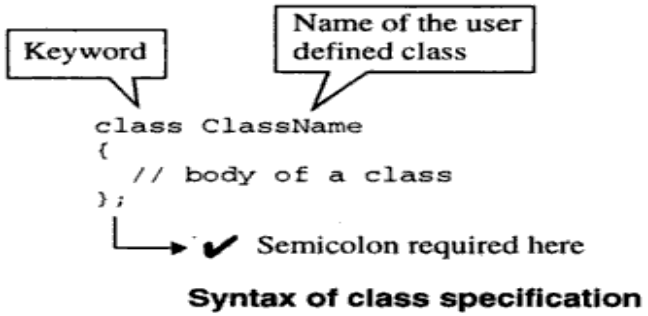
A class is a way to bind the data and its associated functions together. It allows the data (and functions) to be hidden, if necessary, from external use. When defining a class, we are creating a new *abstract data type* that can be treated like any other built-in data type. Generally, a class specification has two parts:

1. Class declaration
2. Class function definitions

The class declaration describes the type and scope of its members. The class function definitions describe how the class functions are implemented.

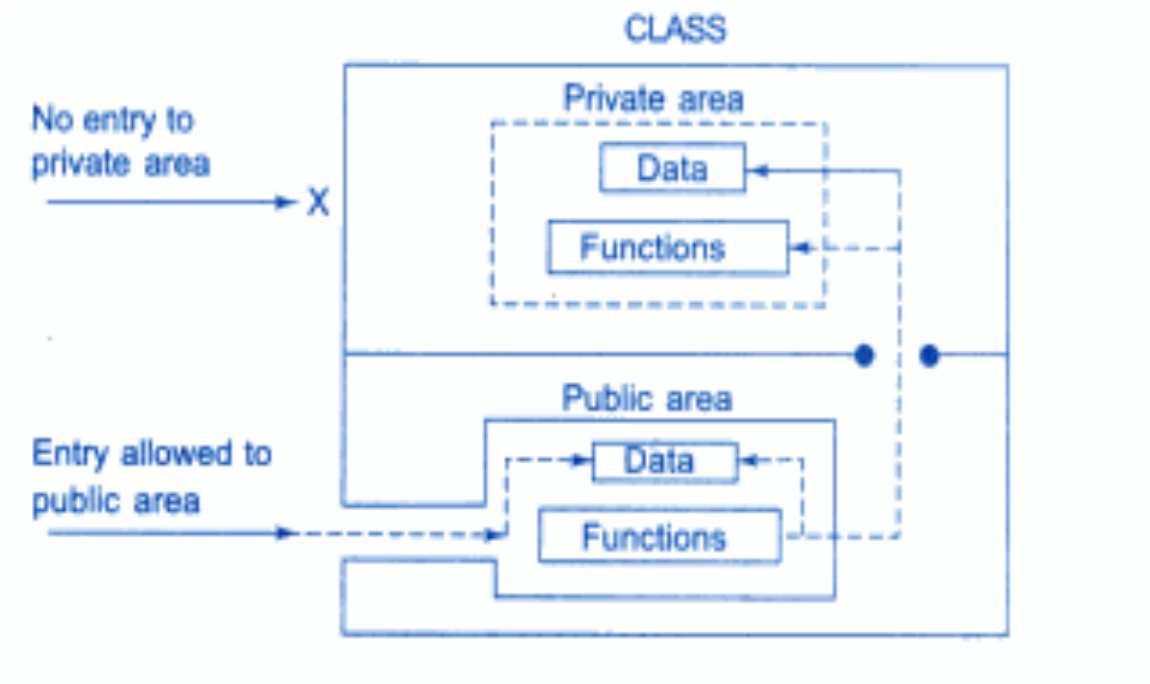
The general form of a class declaration is:

```
class class_name
{
        private:
                variable declarations;
                function declarations;
        public:
                variable declarations;
                function declaration;
};
```



Syntax of class specification

The **class** declaration is similar to a **struct** declaration. The keyword **class** specifies, that what follows is an abstract data of type *class_name*. The body of a class is enclosed within braces and terminated by a semicolon. The class body contains the declaration of variables and functions. These functions and variables are collectively called *class members*. They are usually grouped under two sections, namely, *private* and *public* to denote which of the members are *private* and which of them are *public*. The keywords **private** and **public** are known as visibility labels. Note that these keywords are followed by a colon.

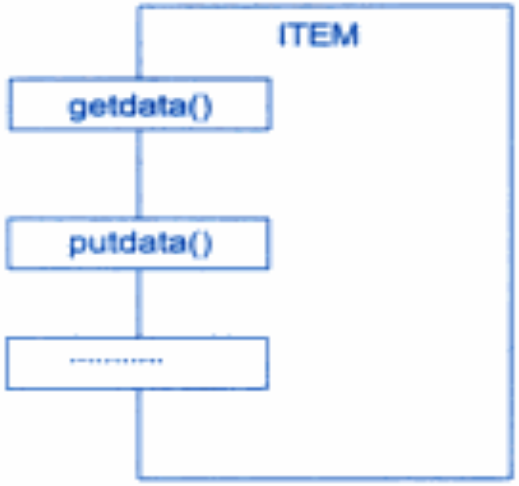


A Simple Class Example

A typical class declaration would look like:

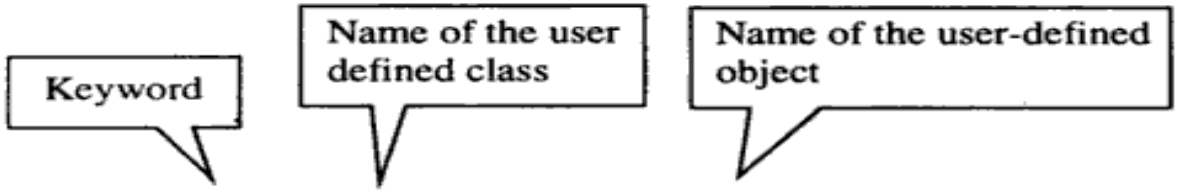
```
class item
{
    int number;                    // variables declaration
    float cost;                    // private by default
  public:
    void getdata(int a, float b);  // functions declaration
    void putdata(void);            // using prototype
  }:// ends with semicolon
```

(a)



(b)

Creating Objects

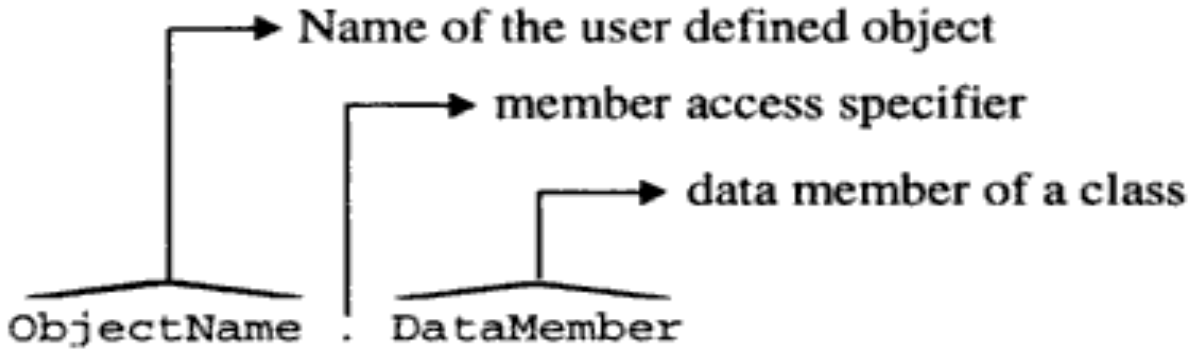



```
class  ClassName ObjectName, ...;
```

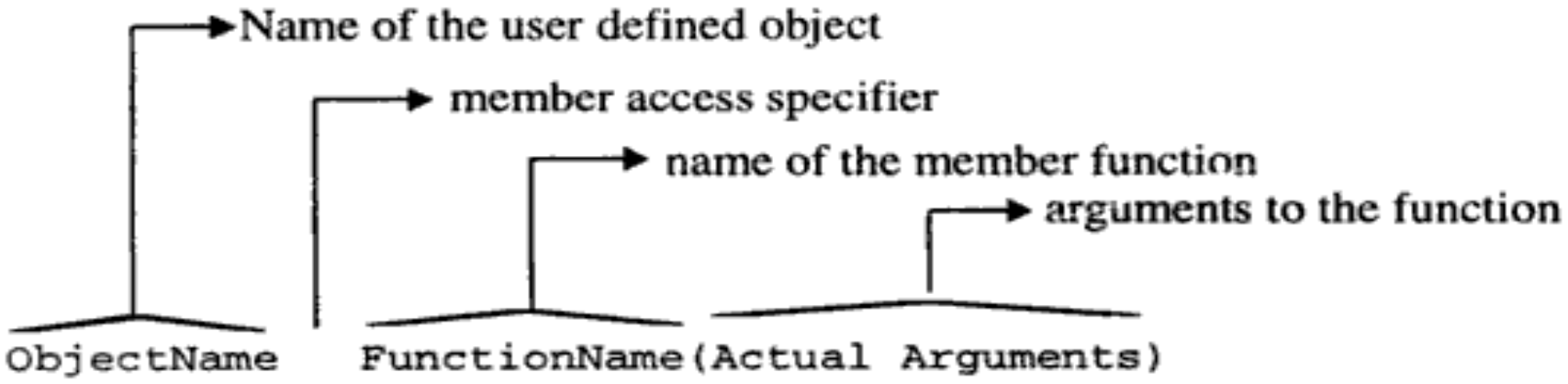
```
item x, y, z;
```

The declaration of an object is similar to that of a variable of any basic type. The necessary memory space is allocated to an object at this stage. Note that class specification, like a structure, provides only a *template* and does not create any memory space for the objects.

Accessing the Members of the Class



(a) Syntax for accessing data member of a class



(b) Syntax for accessing member function of a class

Accessing the Members

```
object-name.function-name (actual-arguments);
```

For example, the function call statement

```
x.getdata(100,75.5);
```

is valid and assigns the value 100 to **number** and 75.5 to **cost** of the object **x** by implementing the **getdata()** function. The assignments occur in the actual function.

Similarly, the statement

```
x.putdata();
```

would display the values of data members. Remember, a member function can be invoked only by using an object (of the same class). The statement like

```
getdata(100,75.5);
```

has no meaning. Similarly, the statement

```
x.number = 100;
```

is also illegal. Although **x** is an object of the type **item** to which **number** belongs, the number (declared private) can be accessed only through a member function and not by the object directly.

It may be recalled that objects communicate by sending and receiving messages. This is achieved through the member functions. For example,

```
x.putdata();
```

sends a message to the object **x** requesting it to display its contents.

Defining Member Functions

Member functions can be defined in two places:

- Outside the class definition.
- Inside the class definition.

Member function definition outside a class declaration

```
class ClassName
{
      ....
      ReturnType MemberFunction(arguments);
      ....
};

ReturnType ClassName :: MemberFunction ( arguments )
{
  // body of the function
}
```

function prototype

user defined class name

Scope resolution operator

Member function definition Outside the class specification

```
return-type class-name :: function-name (argument declaration)
 {
        Function body
 }
```

```
void item :: getdata(int a, float b)
{
      number = a;
      cost = b;
}
```

```
void item :: putdata(void)
{
      cout << "Number :" << number << "\n";
      cout << "Cost   :" << cost   << "\n";
}
```

Inside the Class Definition

Another method of defining a member function is to replace the function declaration by the actual function definition inside the class. For example, we could define the item class as follows:

```
class item
{
    int number;
    float cost;
  public:
    void getdata(int a, float b);      // declaration
         // inline function
    void putdata(void)                 // definition inside the class
    {
         cout << number  <<  "\n";
         cout << cost    <<  "\n";
    }
};
```

When a function is defined inside a class, it is treated as an inline function. Therefore, all the restrictions and limitations that apply to an **inline** function are also applicable here. Normally, only small functions are defined inside the class definition.

Access Specifiers/Visibility Modes:

C++ provides 3 types of Visibility Modes

1. private

2. public

3. protected

Private Members

The private members of a class have strict access control. Only the member functions of the same class can access these members. The private members of a class are inaccessible outside the class. thus. providing a mechanism for preventing accidental modifications of the data members.

```
class Person
{
   private :        Note: colon here
                    access specifier
    // private members
    .....
    int age;        private data
    int getage();   private function
     ....
};

Person p1;
a=p1.age;   ✗ cannot access private data
p1.getage(); ✗ cannot access private function
```

Private members accessibility

```
class Inaccessible
{
    int x;
    void Display()
  {
        cout << "\nData = " << x;
  }
};
void main()
{
    Inaccessible obj1;        // Creating an object.
    obj1.x = 5;               // Error: Invalid access.
    obj1.Display();           // Error: Invalid access.
}
```

The class having all the members with private access control is of no use; there is no means available to communicate with the external world. Therefore, classes of the above type will not contribute anything to the program.

Protected Members

The access control of the protected members is similar to that of private members and has more significance in inheritance.

Access control of protected members is

```
class Person
{
   protected:
    // protected members
    .....
    int age;
    int getage();
     ....
};
Person p1;
a=p1.age;
p1.getage();
```

Note: colon here

access specifier

protected data

protected function

cannot access protected member
(same as private)

Protected members accessibility

Public Members

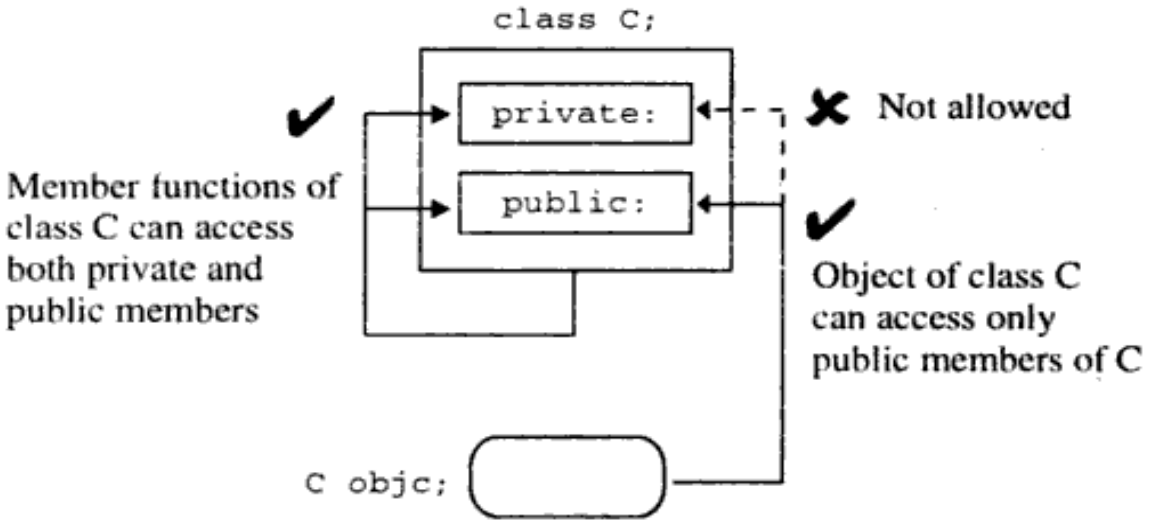
The members of a class, which are to be visible (accessible) outside the class, should be declared in *public* section. All data members and functions declared in the public section of the class can be accessed without any restriction from anywhere in the program, either by functions that belong to the class or by those external to the class. Accessibility control of public members is

```
class Person
{
   public:        Note: colon here
                  access specifier
    // public members
    .....
    int age;      public data
    int getage();  public function
     ....
};
Person p1;
a=p1.age; ✔ can access public data
p1.getage(); ✔ can access public function
```

Public members accessibility

Access Specifier	Accessible to	
	Own class Members	Objects of a Class
`private:`	Yes	No
`protected:`	Yes	No
`public:`	Yes	Yes

Table 10.1: Visibility of class members



Class member accessibility

```
};
//............ Member Function Definition ................
void item :: getdata(int a, float b)      // use membership label
{
      number = a;    // private variables
      cost = b;      // directly used
}
//...................... Main Program ......................

int main()
{
      item x;// create object x

      cout <<  "\nobject x "  <<  "\n";

      x.getdata(100, 299.95);              // call member function
      x.putdata();                         // call member function

      item y;                              // create another object

      cout <<  "\nobject y"  << "\n";

      y.getdata(200, 175.50);
      y.putdata();

      return 0;
 }
```

```
// student.cpp: member functions defined inside the body of the student class
#include <iostream.h>
#include <string.h>
class student
{
   private:
      int roll_no;          // roll number
      char name[ 20 ];      // name of a student
   public:
      // initializing data members
      void setdata( int roll_no_in, char *name_in )
      {
         roll_no = roll_no_in;
         strcpy( name, name_in );
      }
      // display data members on the console screen
      void outdata()
      {
         cout << "Roll No = " << roll_no << endl;
         cout << "Name = " << name << endl;
      }
};
void main()
{
   student s1;         // first object/variable of class student
   student s2;         // second object/variable of class student
   s1.setdata( 1, "Tejaswi" ); // object s1 calls member setdata()
   s2.setdata( 10,"Rajkumar" ); //object s2 calls member setdata()
   cout << "Student details..." << endl;
   s1.outdata();       // object s1 calls member function outdata()
   s2.outdata();       // object s2 calls member function outdata()
}
```

Run

```
Student details...
```

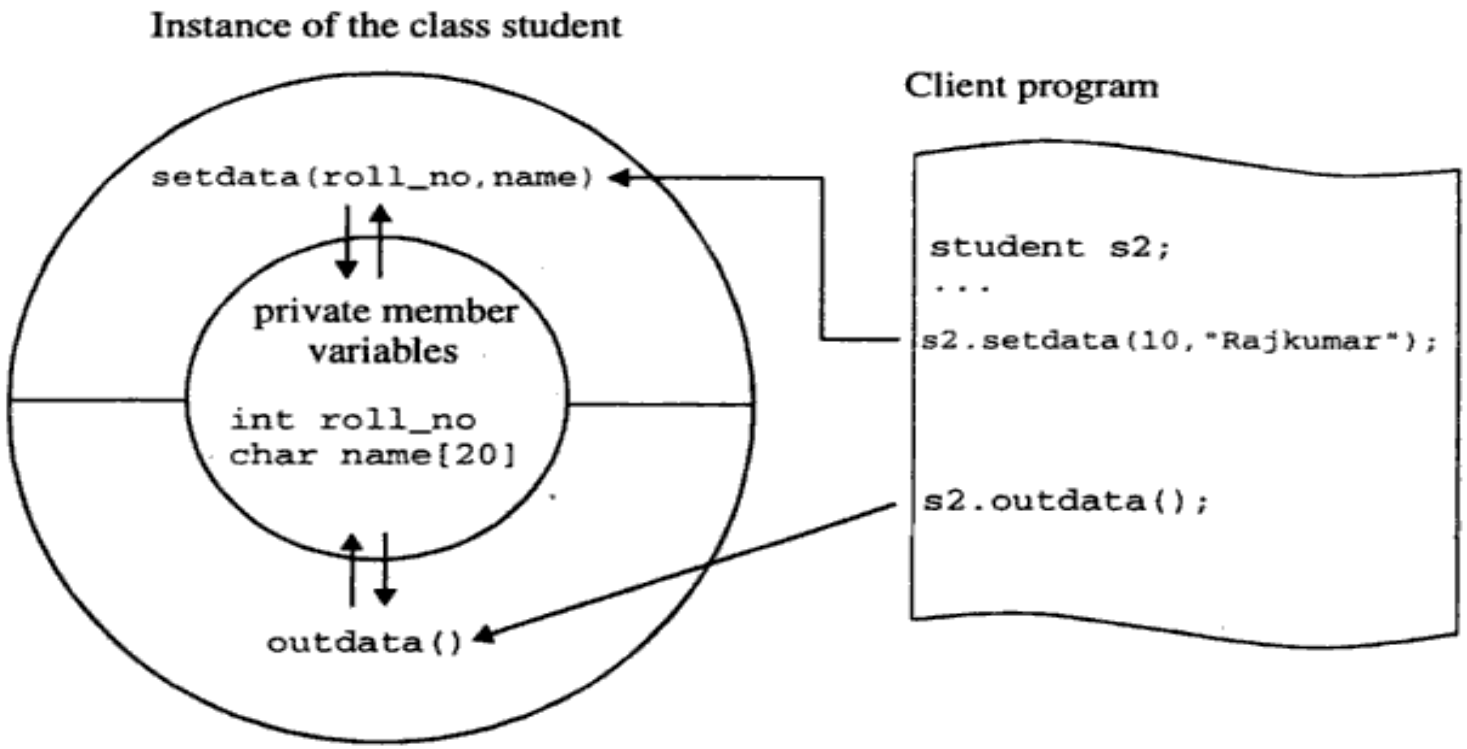
In `main()`, the statements

```
student s1;     // first object/variable of class student
student s2;     // second object/variable of class student
```

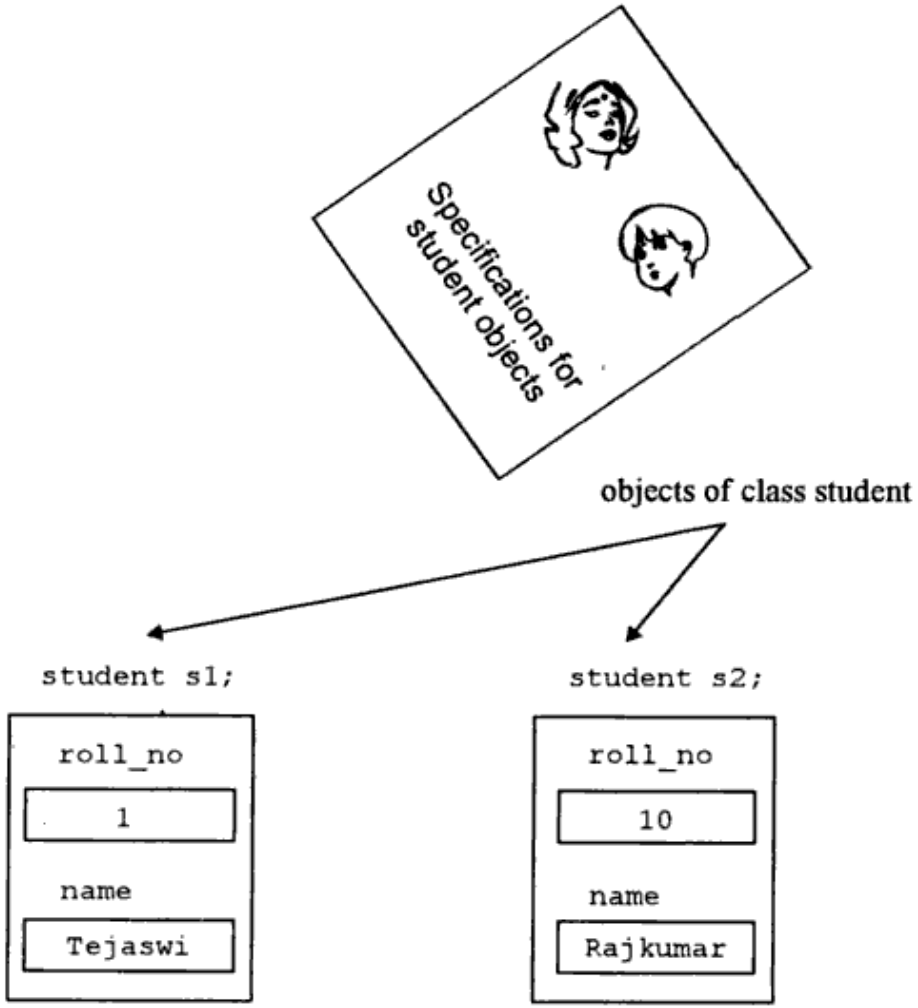
create two objects called `s1` and `s2` of the `student` class. The statements

```
s1.setdata( 1, "Tejaswi" ); //object s1 calls member function setdata
s2.setdata( 10,"Rajkumar" ); //object s2 calls member function setdata
```

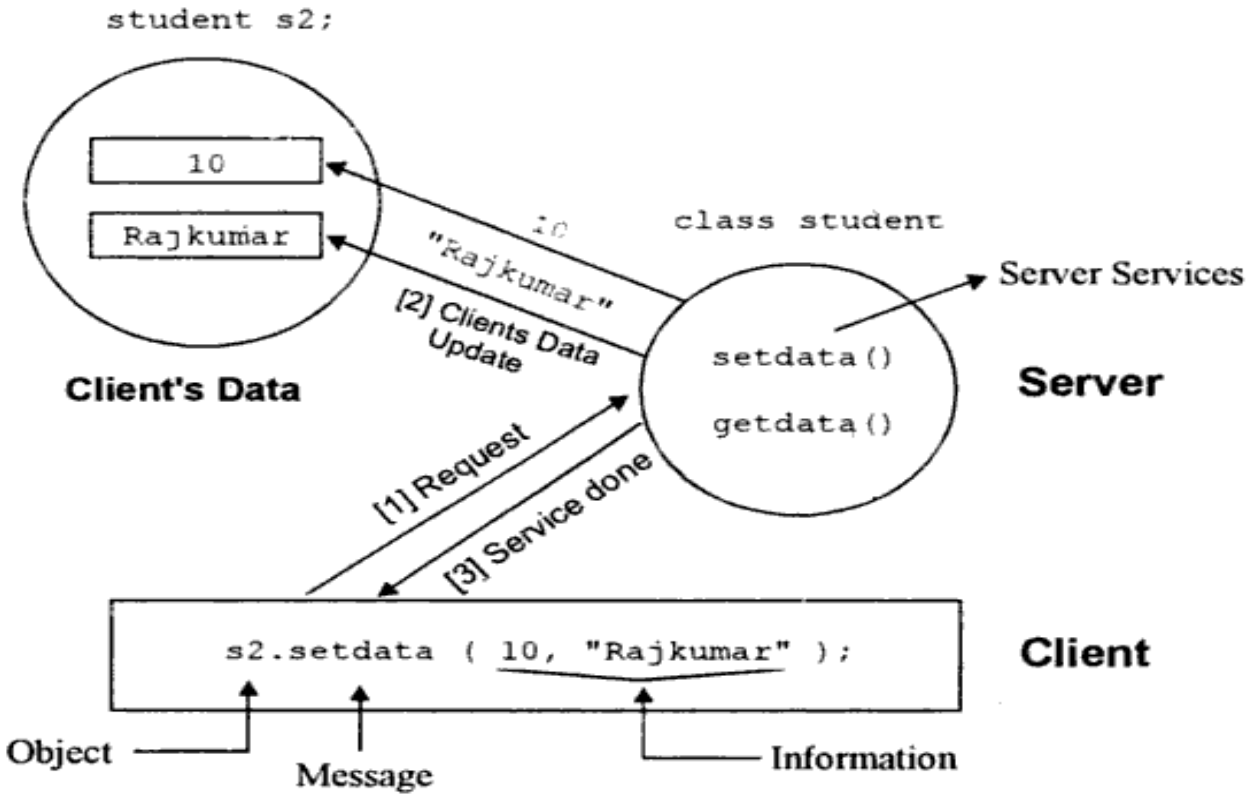
initialize the data members of the objects `s1` and `s2`. The object `s1`'s data member `roll_no` is assigned `1` and `name` is assigned `Tejaswi`. Similarly, the object `s2`'s data member `roll_no` is assigned `10` and `name` is assigned `Rajkumar`.



Two objects of the class student



Client-Server model for message communication



Characteristics of Member Functions:

- A program can have several classes and they can have member functions with the same name. The ambiguity of the compiler in deciding *which function belongs to which class* can be resolved by the use of membership label (`ClassName::`), the scope resolution operator.
- Private members of a class, can be accessed by all the members of the class, whereas non-member functions are not allowed to access. However, friend functions can access them.
- Member functions of the same class can access all other members of their own class without the use of dot operator.
- Member functions defined as `public` act as an interface between the service provider (server) and the service seeker (client).
- A class can *have* multiple member functions with the same name as long as they differ in terms of argument specification (data type or number of arguments).

In OOPs, the process of programming involves the following steps:

- **Creation of classes for defining objects and their behaviors.**
- **Creation of class objects; class declaration acts like a blueprint for which physical resources are not allocated.**
- **Establishment of communication among objects through message passing**

Write a simple program using class in C++ to input subject mark

```
class marks
{
private :
int roll; int ml,m2;
public:
void getdata(); void
displaydata();
};
void marks: :getdata()
{ cout<<“enter the roll-no:”;cin>>roll;
cout<<”enter 1st subject mark:”; cin>>ml;
cout<<”enter 2nd subject mark:”; cin>>m2;
}
void marks: :displaydata()
{ cout<<“Roll No.”<<roll;
cout<<”Ist subject mark:”<<ml<<endl ; cout<<”2nd
subject mark:”<<m2;
}
```

```
void main()
{
clrscr();
marks x;
x.getdata();
x.displaydata();
}
```

Nesting of Member Functions

```
// nesting.cpp: A member function accessing another member function
#include <iostream.h>
class NumberPairs
{
   int num1, num2;       // private by default
   public:
      void read()
      {
         cout << "Enter First Number: ";
         cin >> num1;
         cout << "Enter Second Number: ";
         cin >> num2;
      }
      int max()       // member function
      {
         if( num1 > num2 )
            return num1;
         else
            return num2;
      }
      // Nesting of member function
      void ShowMax()
      {
         // calls member function max()
         cout << "Maximum = " << max();
      }
};
void main()
{
   NumberPairs n1;
   n1.read();
   n1.ShowMax();
}
```

Run

```
Enter First Number: 5
Enter Second Number: 10
Maximum = 10
```

Private Member Functions

A private member function can only be called by another function that is a member of its class. Even an object cannot invoke a private function using the dot operator. Consider a class as defined below:

```
class sample
{
      int m;
      void read(void);          // private member function
  public:
      void update(void);
      void write(void);
};
```

If **s1** is an object of **sample**, then

```
s1.read();        // won't work; objects cannot access
                  // private members
```

is illegal. However, the function **read()** can be called by the function **update()** to update the value of **m**.

```
void sample :: update(void)
{
      read();     // simple call; no object used
}
```

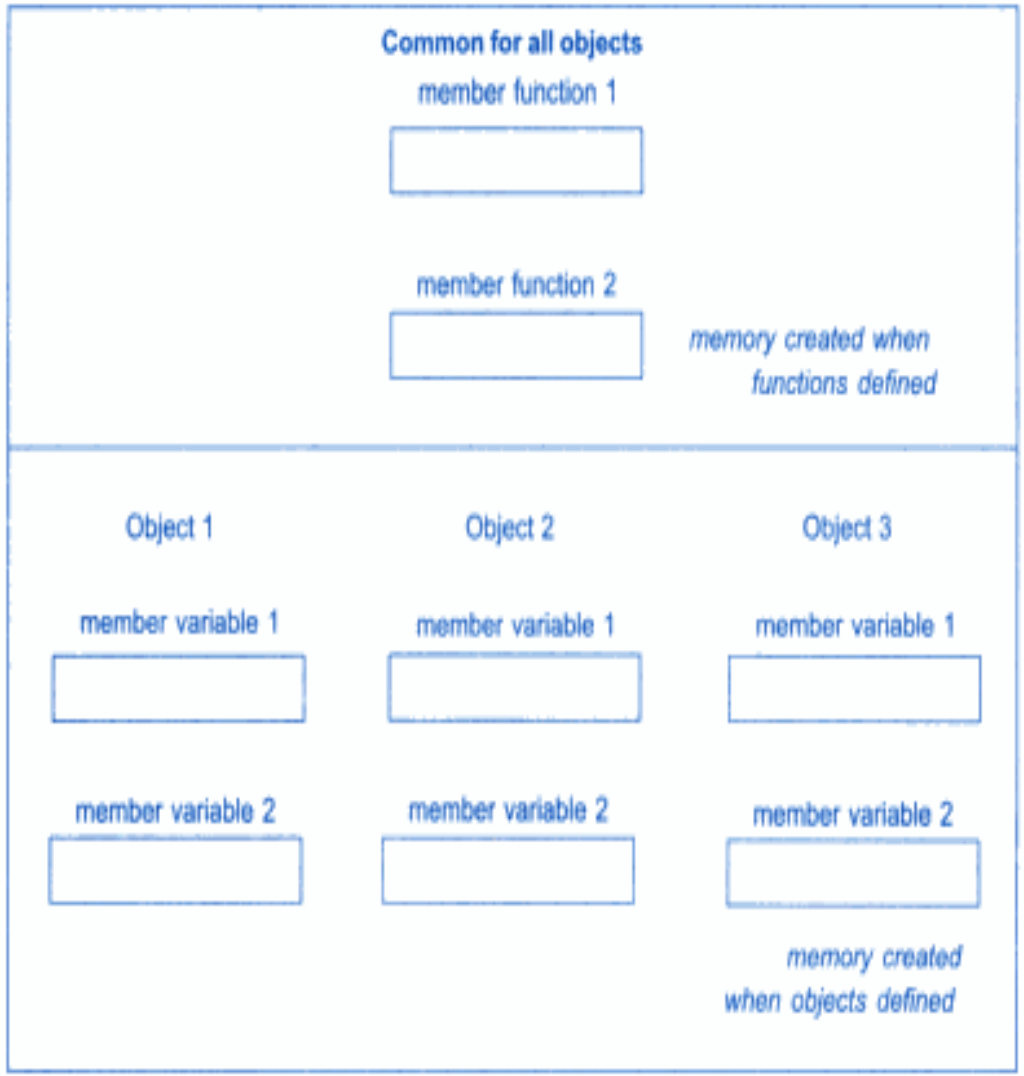
Arrays within a Class

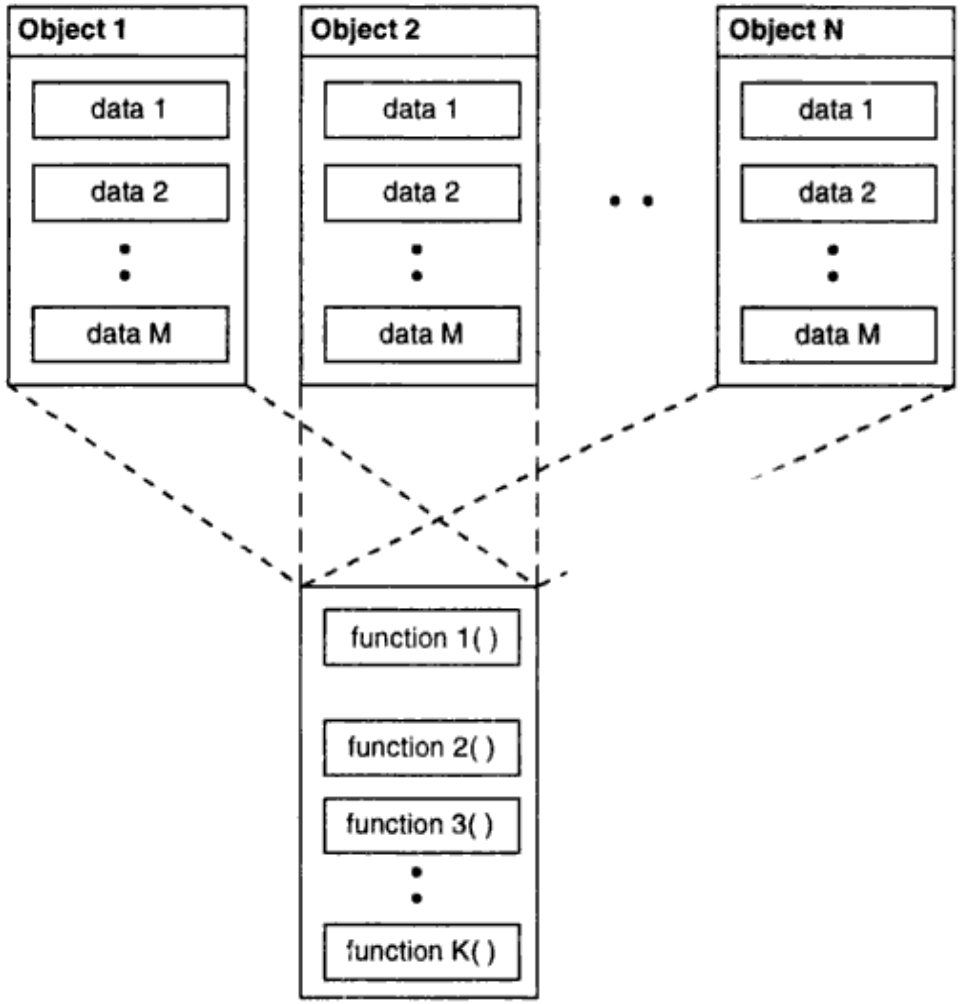
The arrays can be used as member variables in a class. The following class definition is valid.

```
const int size=10;      // provides value for array size

class array
{
     int a[size];       // 'a' is int type array
  public:
     void setval(void);
     void display(void);
};
```

The array variable **a[ ]** declared as a private member of the class **array** can be used in the member functions, like any other array variable. We can perform any operations on it. For instance, in the above class definition, the member function **setval()** sets the values of elements of the array **a[ ]**, and **display()** function displays the values. Similarly, we may use other member functions to perform any other operations on the array values.

Memory Allocation for Objects





Separate memory for object's data members
Shared memory for class functions

Static Data Members

A data member of a class can be qualified as static. The properties of a **static** member variable are similar to that of a C static variable. A static member variable has certain special characteristics. These are :

- It is initialized to zero when the first object of its class is created. No other initialization is permitted.
- Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
- It is visible only within the class, but its lifetime is the entire program.

Static variables are normally used to maintain values common to the entire class. For example, a static data member can be used as a counter that records the occurrences of all the objects. Program 5.4 illustrates the use of a static data member.

STATIC CLASS MEMBER

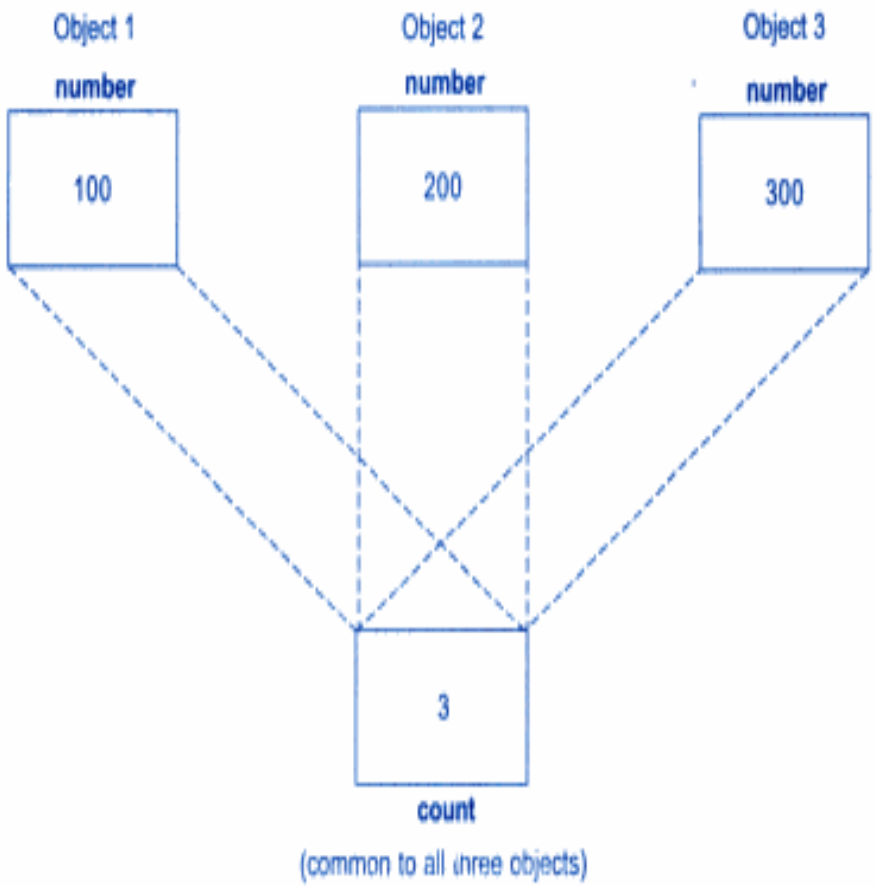
```
#include <iostream>

using namespace std;

class item
{
        static int count;
        int number;
  public:
        void getdata(int a)
        {
                number = a;
                count ++;
        }
        void getcount(void)
        {
                cout << "count: ";
                cout << count  << "\n";
        }
};

int item :: count;

int main()
{

    item a, b, c;          // count is initialized to zero
    a.getcount();          // display count
    b.getcount();
    c.getcount();

    a.getdata(100);        // getting data into object a
    b.getdata(200);        // getting data into object b
    c.getdata(300);        // getting data into object c

    cout << "After reading data" << "\n";

    a.getcount();          // display count
    b.getcount();
    c.getcount();
    return 0;
}
```

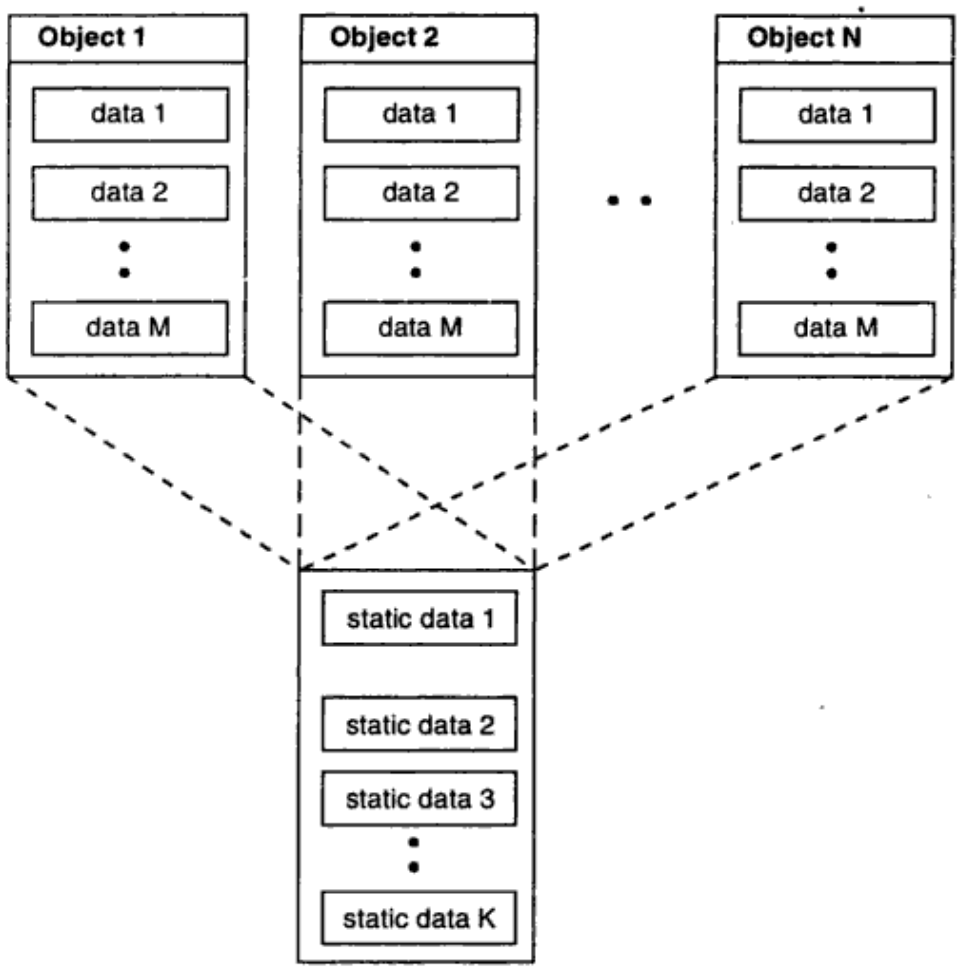
Output of Program

```
count: 0
count: 0
count: 0
After reading data
count: 3
count: 3
count: 3
```

```
int item :: count;     // definition of static data member
```

Memory allocation for static member





Separate memory for class's automatic data members
Shared memory for class's static data members

Static Member Functions

Like **static** member variable, we can also have **static** member functions. A member function that is declared **static** has the following properties:

- A **static** function can have access to only other static members (functions or variables) declared in the same class.
- A **static** member function can be called using the class name (instead of its objects) as follows:

```
class-name :: function-name;
```

STATIC MEMBER FUNCTION

```
#include <iostream>

using namespace std;

class test
{
        int code;
        static int count;            // static member variable
public:
        void setcode(void)
        {
        code = ++count;
        }
        void showcode(void)
        {
        cout << "object number: " << code << "\n";
        }
        static void showcount(void)     // static member function
        {
        cout << "count: " << count << "\n";
        }
};

int test :: count;
int main()
{
        test t1, t2;

        t1.setcode();
        t2.setcode();

        test :: showcount();    // accessing static function

        test t3;
        t3.setcode();

        test :: showcount();

        t1.showcode();
        t2.showcode();
        t3.showcode();

        return 0;
}
```

Output of Program

```
count: 2
count: 3
object number: 1
object number: 2
object number: 3
```

Remember, the following function definition will not work:

```
static void showcount()
{
    cout << code;     // code is not static
}
```

Array of objects

```
#include <iostream>

using namespace std;

class employee

{
        char  name[30];       // string as class member
        float age;
  public:
        void getdata(void);
        void putdata(void);
};
void employee :: getdata(void)
{
        cout << "Enter name: ";
        cin  >> name;
        cout << "Enter age:  ";
        cin  >> age;
}
void employee :: putdata(void)
{
        cout << "Name: " << name << "\n";
        cout << "Age:  " << age  << "\n";
}
const int size=3;
int main()
{
        employee manager[size];
        for(int i=0; i<size; i++)
        {
                cout << "\nDetails of manager" << i+1 << "\n";
                manager[i].getdata();
        }
        cout <<  "\n";
        for(i=0; i<size; i++)
        {
                cout << "\nManager" << i+1 << "\n";
                manager[i].putdata();
        }
        return 0;
 }
```

Output of Program

Interactive input

```
Details of manager1
Enter name: xxx
Enter age:  45

Details of manager2
Enter name: yyy
Enter age:  37

Details of manager3
Enter name: zzz
Enter age:  50
```

Program output

```
Manager1
Name: xxx
Age:  45

Manager2
Name: yyy
Age:  37

Manager3
Name: zzz
Age:  50
```

Objects as Function Arguments

It is possible to have functions which accept objects of a class as arguments, just as there are functions which accept other variables as arguments. Like any other data type, an object can be passed as an argument to a function by the following ways:

- pass-by-value, a copy of the entire object is passed to the function
- pass-by-reference, only the address of the object is passed implicitly to the function
- pass-by-pointer, the address of the object is passed explicitly to the function

In the case of *pass-by-value*, a copy of the object is passed to the function and any modifications made to the object inside the function is not reflected in the object used to call the function. Whereas, in *pass-by-reference* or *pointer*, an address of the object is passed to the function and any changes made to the object inside the function is reflected in the actual object. The parameter passing by reference or pointer is more efficient since, only the address of the object is passed and not a copy of the entire object.

Objects as Function Arguments

Output of Program

```
T1 = 2 hours and 45 minutes
T2 = 3 hours and 30 minutes
T3 = 6 hours and 15 minutes
```

```
#include <iostream>

using namespace std;

class time
{
        int  hours;
        int  minutes;
  public:
        void gettime(int h, int m)
        {  hours = h; minutes = m; }
        void puttime(void)
        {
                cout << hours << " hours and ";
                cout << minutes << " minutes " << "\n";
        }
        void sum(time, time);  // declaration with objects as arguments
};
void time :: sum(time t1, time t2)          // t1, t2 are objects
{
        minutes = t1.minutes + t2.minutes;
        hours = minutes/60;
        minutes = minutes%60;
        hours = hours + t1.hours + t2.hours;
}
int main()
{
        time T1, T2, T3;

        T1.gettime(2,45);     // get T1
        T2.gettime(3,30);     // get T2

        T3.sum(T1,T2);// T3=T1+T2

        cout << "T1 = "; T1.puttime();       // display T1
        cout << "T2 = "; T2.puttime();       // display T2
        cout << "T3 = "; T3.puttime();       // display T3

        return 0;
}
```

note

Since the member function **sum( )** is invoked by the object **T3**, with the objects **T1** and **T2** as arguments, it can directly access the hours and minutes variables of **T3**. But, the members of **T1** and **T2** can be accessed only by using the dot operator (like **T1.hours** and **T1.minutes**). Therefore, inside the function sum(), the variables **hours** and **minutes** refer to **T3**, **T1.hours** and **T1.minutes** refer to **T1**, and **T2.hours** and **T2.minutes** refer to **T2**.

Figure 5.6 illustrates how the members are accessed inside the function **sum()**.

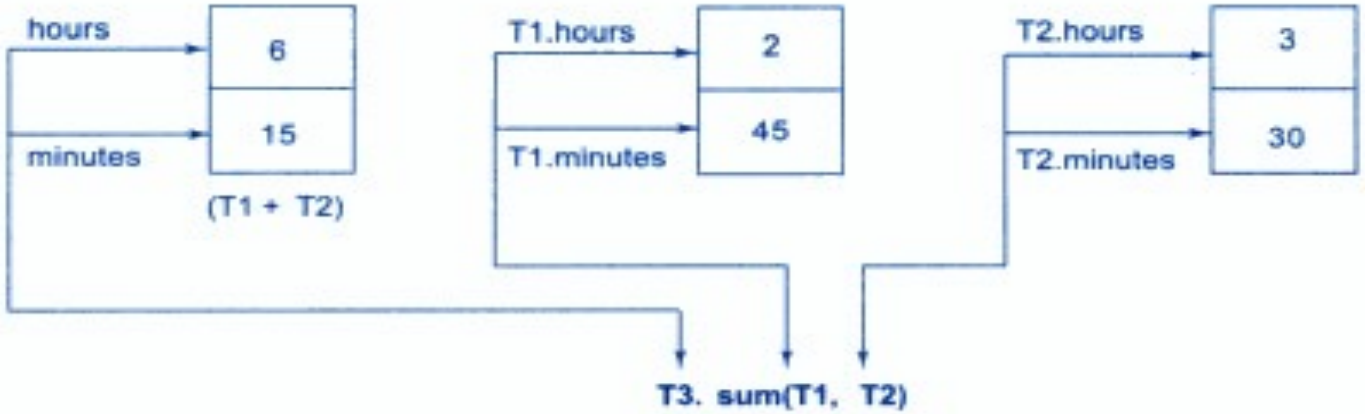


Fig. 5.6 ⇔ *Accessing members of objects within a called function*

An object can also be passed as an argument to a non-member function. However, such functions can have access to the **public member** functions only through the objects passed as arguments to it. These functions cannot have access to the private data members.

Passing Objects by Value

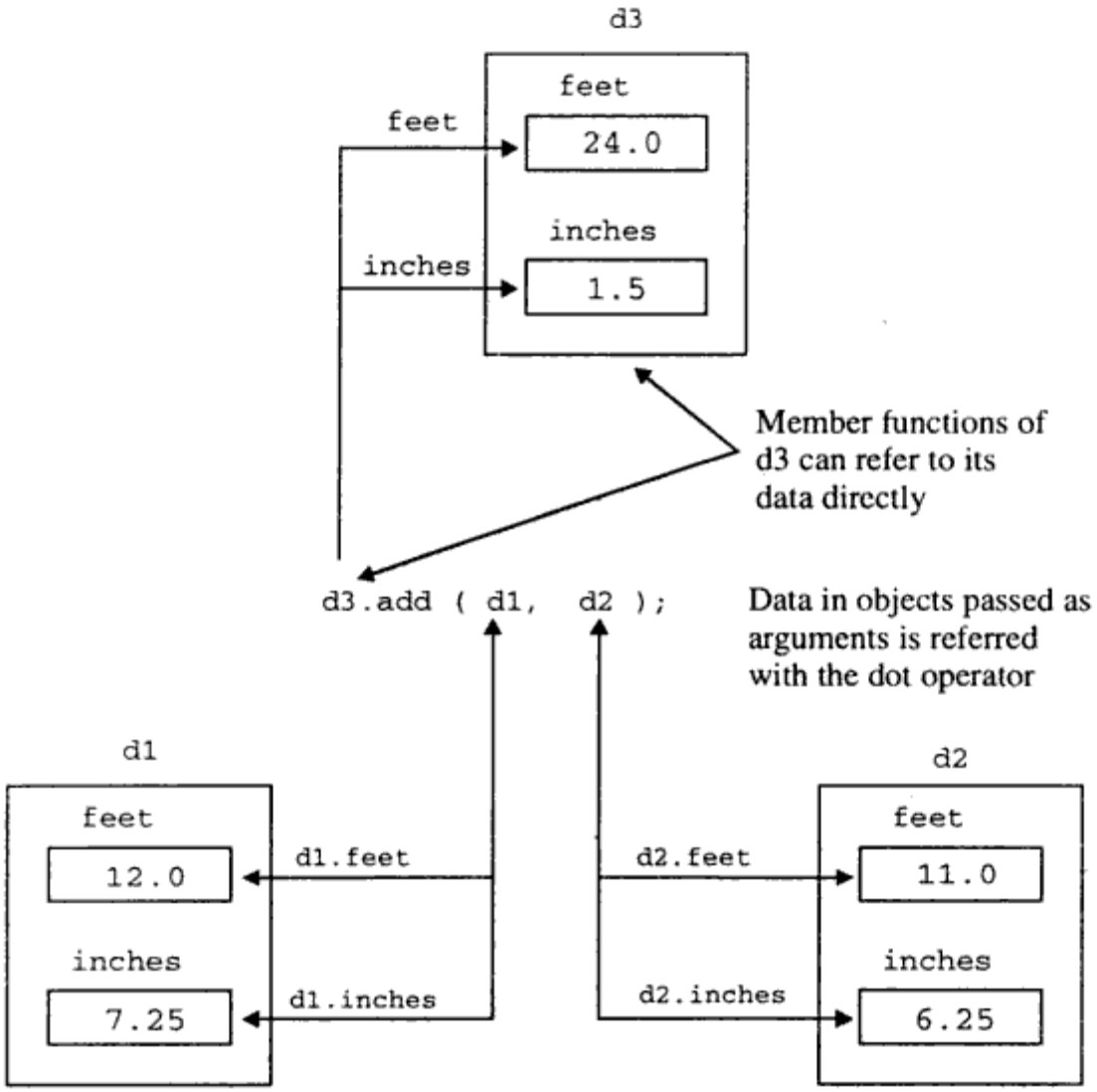
The program `distance.cpp` illustrates the use of objects as function arguments in *pass-by-value* mechanism. It performs the addition of distance in feet and inches format.

```
// distance.cpp: distance manipulation in feet and inches
#include <iostream.h>
class distance
{
   private:
      float feet;
      float inches;
   public:
      void init( float ft, float in )
      {
         feet = ft;
         inches = in;
      }
      void read()
      {
         cout << "Enter feet: ";    cin >> feet;
         cout << "Enter inches: ";  cin >> inches;
      }
```

```
        void show()
        {
           cout << feet << "-" << inches << '\"';
        }
        void add( distance d1, distance d2 )
        {
           feet = d1.feet + d2.feet;
           inches = d1.inches + d2.inches;
           if( inches >= 12.0 )
           {
              // 1 foot = 12.0 inches
              feet = feet + 1.0;
              inches = inches - 12.0;
           }
        }
};
void main()
{
   distance d1, d2, d3;
   d2.init( 11.0, 6.25 );
   d1.read();
   cout << "d1 = ";  d1.show();
   cout << "\nd2 = ";   d2.show();
   d3.add( d1, d2 ); // d3 = d1 + d2
   cout << "\nd3 = d1+d2 = ";  d3.show();
}
```

Run

```
Enter feet: 12.0
Enter inches: 7.25
d1 = 12'-7.25"
d2 = 11'-6.25"
d3 = d1+ d2 = 24'-1.5"
```



Objects of the distance class as parameters

Passing Objects by Reference

Accessibility of the objects passed by reference is similar to those passed by value. Modifications carried out on such objects in the called function will also be reflected in the calling function. The method of passing objects as reference parameters to a function is illustrated in the program account.cpp. Given the account numbers and the balance of two accounts, this program transfers a specified sum from one of these accounts to the other and then, updates the balance in both the accounts.

```
// account.cpp: passing objects as parameters to functions
#include<iostream.h>
class AccClass
{
   private:                    // class data members
      int accno;
      float balance;
   public:                     // class function members
      void getdata()
      {
         cout << "Enter the account number for acc1 object: ";
         cin >> accno;
         cout << "Enter the balance: ";
         cin >> balance;
      }
```

```
    void setdata( int accIn )
    {
       accno = accIn;
       balance = 0;
    }
    void setdata( int accIn, float balanceIn )
    {
       accno = accIn;
       balance = balanceIn;
    }
    void display()
    {
       cout << "Account number is: " << accno << endl;
       cout << "Balance is: " << balance << endl;
    }
    void MoneyTransfer( AccClass & acc, float amount );
};
// acc1.MoneyTransfer( acc2, 100 ),transfers 100 rupees from acc1 to acc2
void AccClass::MoneyTransfer( AccClass & acc, float amount )
{
   balance = balance - amount;   // deduct money from source
   acc.balance = acc.balance + amount; // add money to destination
}
```

```
void main()
{
   int trans_money;
   AccClass acc1, acc2, acc3;
   acc1.getdata();
   acc2.setdata( 10 );
   acc3.setdata( 20, 750.5 );
   cout << "Account Information..." << endl;
   acc1.display();
   acc2.display();
   acc3.display();
   cout << "How much money is to be transferred from acc3 to acc1: ";
   cin >> trans_money;
   acc3.MoneyTransfer(acc1,trans_money); //transfers money from acc3 to acc1
   cout << "Updated Information about accounts..." << endl;
   acc1.display();
   acc2.display();
   acc3.display();
}
```

Run

```
Enter the account number for acc1 object: 1
Enter the balance: 100
Account Information...
Account number is: 1
Balance is: 100
Account number is: 10
Balance is: 0
```

```
Account number is: 20
Balance is: 750.5
How much money is to be transferred from acc3 to acc1: 200
Updated Information about accounts...
Account number is: 1
Balance is: 300
Account number is: 10
Balance is: 0
Account number is: 20
Balance is: 550.5
```

In `main()`, the statement

```
acc3.MoneyTransfer( acc1, trans_money );
```

transfers the object `acc1` by reference to the member function `MoneyTransfer()`. It is to be noted that when the `MoneyTransfer()` is invoked with `acc1` as the object parameter, the data members of `acc3` are accessed without the use of the class member access operator, while the data members of `acc1` are accessed by using their names in association with the name of the object to which they belong. An object can also be passed to a non-member function of the class and that can have access to the public members only through the objects passed as arguments to it.

Passing Objects by Pointer

The members of objects passed by pointer are accessed by using the -> operator, and they have similar effect as those passed by value. The above program requires the following changes if parameters are to be passed by pointer:

1. The prototype of the member function `MoneyTransfer()` has to be changed to:

```
void MoneyTransfer( AccClass * acc, float amount );
```

2. The definition of the member function `MoneyTransfer()` has to be changed to:

```
void AccClass::MoneyTransfer( AccClass & acc, float amount )
{
   balance = balance - amount;   // deduct money from source
   acc->balance = acc->balance + amount; // add money to destination
}
```

3. The statement invoking the member function `MoneyTransfer()` has to be changed to:

```
acc3.MoneyTransfer( &acc1, trans_money );
```

10.13 Returning Objects from Functions

Similar to sending objects as parameters to functions, it is also possible to return objects from functions. The syntax used is similar to that of returning variables from functions. The return type of the function is declared as the return object type. It is illustrated in the program `complex.cpp`.

```
// complex.cpp: Addition of Complex Numbers, class complex as data type
#include <iostream.h>
#include <math.h>

class complex
{
   private:
      float real;    // real part of complex number
      float imag;    // imaginary part of complex number
```

```
   public:
      void getdata()
      {
         cout << "Real Part ? ";
         cin >> real;
         cout << "Imag Part ? ";
         cin >> imag;
      }
      void outdata( char *msg )     // display number in x+iy form
      {
         cout << msg << real;
         if( imag < 0 )
            cout << "-i";
         else
            cout << "+i";
         cout << fabs(imag) << endl;
      }
      complex add( complex c2 );    // addition of complex numbers
};
complex complex::add( complex c2 )  // add default and c2 objects
{
   complex temp;                    // object temp of complex class
   temp.real = real + c2.real;      // add real parts
   temp.imag = imag + c2.imag;      // add imaginary parts
   return( temp );                  // return complex object
}
```

```
void main()
{
   complex c1, c2, c3;      // c1, c2, and c2 are objects of complex
   cout << "Enter Complex Number c1 .." << endl;
   c1.getdata();
   cout << "Enter Complex Number c2 .." << endl;
   c2.getdata();
   c3 = c1.add( c2 );       // add c1 and c2 assign to c3
   c3.outdata( "c3 = c1.add( c2 ): ");
}
```

Run

```
Enter Complex Number c1 ..
Real Part ? 1.5
Imag Part ? 2
Enter Complex Number c2 ..
Real Part ? 3
Imag Part ? -4.3
c3 = c1.add( c2 ): 4.5-i2.3
```

In `main()`, the statement

```
c3 = c1.add( c2 );      // add c1 and c2 assign to c3
```

invokes the function `add()` of the class `complex` by passing the object `c2` as a parameter. The statement in this function,

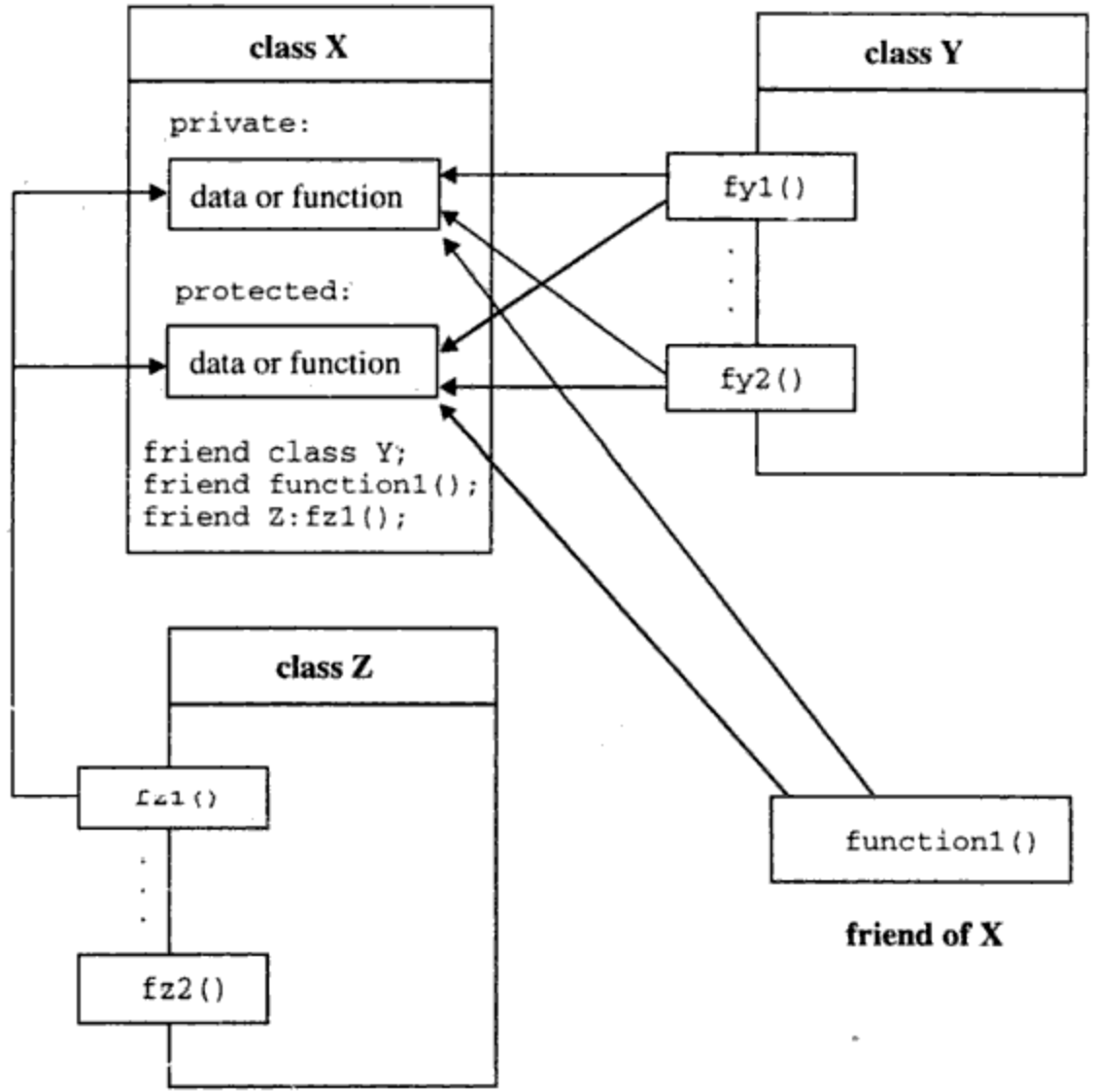
```
return( temp );       // return complex object
```

returns the object `temp` as a return object.

Friendly Functions

The concept of encapsulation and data hiding dictate that non-member functions should not be allowed to access an object's private and protected members. The policy is, *if you are not a member you cannot get it*. Sometimes this feature leads to considerable inconvenience in programming. Imagine that the user wants a function to operate on objects of two different classes. At such times, it is required to allow functions outside a class to access and manipulate the private members of the class. In C++, this is achieved by using the concept of *friends*.

One of the convenient and a controversial feature of C++ is allowing non-member functions to access even the private members of a class using friend functions or friend classes. It permits a function or all the functions of another class to access a different class's private members.



Class members accessibility in various forms

To make an outside function "friendly" to a class, we have to simply declare this function as a **friend** of the class as shown below:

```
class ABC
{
        .....
        .....
  public:
        .....
        .....
        friend void xyz(void);   // declaration
};
```

The function declaration should be preceded by the keyword **friend**. The function is defined elsewhere in the program like a normal C++ function. The function definition does not use either the keyword **friend** or the scope operator ::. The functions that are declared with the keyword friend are known as friend functions. A function can be declared as a **friend** in any number of classes. A friend function, although not a member function, has full access rights to the private members of the class.

A friend function possesses certain special characteristics:

- It is not in the scope of the class to which it has been declared as **friend**.
- Since it is not in the scope of the class, it cannot be called using the object of that class.
- It can be invoked like a normal function without the help of any object.
- Unlike member functions, it cannot access the member names directly and has to use an object name and dot membership operator with each member name.(e.g. A.x).
- It can be declared either in the public or the private part of a class without affecting its meaning.
- Usually, it has the objects as arguments.

```
#include <iostream>

using namespace std;

class sample

    {
        int a;
        int b;
    public:
        void setvalue() {a=25; b=40; }
        friend float mean(sample s);
    };
    float mean(sample s)
    {
        return float(s.a + s.b)/2.0;
    }

    int main()
    {
        sample X;       // object X
        X.setvalue();
        cout << "Mean value = " << mean(X) << "\n";

        return 0;
    }
```

note

The friend function accesses the class variables **a** and **b** by using the dot operator and the object passed to it. The function call **mean(X)** passes the object **X** by value to the friend function.

Member functions of one class can be **friend** functions of another class. In such cases, they are defined using the scope resolution operator as shown below:

```
class X
{
      .....
      .....
      int fun1();          // member function of X
      .....
};

class Y
{

      .....
      .....
      friend int X :: fun1();          // fun1() of X
                                       // is friend of Y
      .....
};
```

The function **fun1()** is a member of **class X** and a **friend** of class **Y**.

We can also declare all the member functions of one class as the friend functions of another class. In such cases, the class is called a **friend class**. This can be specified as follows:

```
class Z
{
      .....
      friend class X;     // all member functions of X are
                          // friends to Z
};
```

A FUNCTION FRIENDLY TO TWO CLASSES

```
#include <iostream>

using namespace std;

class ABC;      // Forward declaration
//-------------------------------------------------------//
class XYZ
{
        int x;
  public:
        void setvalue(int i) {x = i;}
        friend void max(XYZ, ABC);
};
//-------------------------------------------------------//
class ABC
{
        int a;
  public:
```

```
//-------------------------------------------------------//
void max(XYZ m, ABC n)      // Definition of friend
{
        if(m.x >= n.a)
                cout << m.x;
        else
                cout << n.a;
}
//-------------------------------------------------------//
int main()
{
        ABC abc;
        abc.setvalue(10);
        XYZ xyz;
        xyz.setvalue(20);
        max(xyz, abc);

        return 0;
}
```

note

The function max() has arguments from both **XYZ** and **ABC**. When the function max() is declared as a friend in **XYZ** for the first time, the compiler will not acknowledge the presence of ABC unless its name is declared in the beginning as

```
class ABC;
```

This is known as 'forward' declaration.

Returning Objects

```
#include <iostream>

using namespace std;

class complex                    // x + iy form
{
    float x;                     // real part
    float y;                     // imaginary part
  public:
    void input(float real, float imag)
    { x = real; y = imag; }
```

```
        friend complex sum(complex, complex);

        void show(complex);
    };

    complex sum(complex c1, complex c2)
    {
        complex c3;              // objects c3 is created
        c3.x = c1.x + c2.x;
        c3.y = c1.y + c2.y;
        return(c3);              // returns object c3
    }

    void complex :: show(complex c)
    {
        cout << c.x << " + j" << c.y << "\n";
    }

    int main()
    {
        complex A, B, C;

        A.input(3.1, 5.65);
        B.input(2.75, 1.2);

        C = sum(A, B);           // C = A + B

        cout << "A = "; A.show(A);
        cout << "B = "; B.show(B);
        cout << "C = "; C.show(C);

        return 0;
    }
```

Output of Program

```
A = 3.1 + j5.65
B = 2.75 + j1.2
C = 5.85 + j6.85
```

const Member Functions

Certain member functions of a class, access the class data members without modifying them. It is advisable to declare such functions as `const` (constant) functions. The syntax for declaring `const` member functions is shown . A `const` member function is used to indicate that it does not alter the data fields of the object, but only inspects them.

Keyword

```
ReturnType FunctionName(arguments) const
```

Syntax of declaring a constant member function

If a member function does not alter any data in the class, then we may declare it as a **const** member function as follows:

```
void mul(int, int) const;
double get_balance() const;
```

The qualifier **const** is appended to the function prototypes (in both declaration and definition). The compiler will generate an error message if such functions try to alter the data values.

Pointers to Members

It is possible to take the address of a member of a class and assign it to a pointer. The address of a member can be obtained by applying the operator & to a "fully qualified" class member name. A class member pointer can be declared using the operator ::* with the class name. For example, given the class

```
class A
{
  private:
     int m;
  public:
     void show();
};
```

We can define a pointer to the member m as follows:

```
int A::* ip = &A :: m;
```

The **ip** pointer created thus acts like a class member in that it must be invoked with a class object. In the statement above, the phrase **A::*** means "pointer-to-member of **A** class". The phrase **&A::m** means the "address of the m member of A class".

Remember, the following statement is not valid:

```
int *ip = &m;     // won't work
```

This is because **m** is not simply an **int** type data. It has meaning only when it is associated with the class to which it belongs. The scope operator must be applied to both the pointer and the member.

The pointer **ip** can now be used to access the member **m** inside member functions (or friend functions). Let us assume that **a** is an object of **A** declared in a member function. We can access **m** using the pointer **ip** as follows:

```
cout << a.*ip;   // display
cout << a.m;     // same as above
```

Now, look at the following code:

```
ap = &a;                  // ap is pointer to object a
cout << ap -> *ip;        // display m
cout << ap -> m; // same as above
```

The *dereferencing operator* ->* is used to access a member when we use pointers to both the object and the member. The *dereferencing* operator.* is used when the object itself is used with the member pointer. Note that ***ip** is used like a member name.

We can also design pointers to member functions which, then, can be invoked using the dereferencing operators in the **main** as shown below :

```
(object-name .* pointer-to-member function) (10);
(pointer-to-object ->* pointer-to-member function) (10)
```

The precedence of () is higher than that of .* and ->*, so the parentheses are necessary.

```
#include <iostream>

using namespace std;

class M
{
        int x;
        int y;
 public:
        void set_xy(int a, int b)
        {
                x = a;
                y = b;
        }
        friend int sum(M m);

};
int sum(M m)
{
        int M ::* px = &M :: x;
        int M ::* py = &M :: y;
        M *pm = &m;
        int S = m.*px + pm->*py;
        return S;
}

int main()
{
        M n;
        void (M :: *pf)(int,int) = &M :: set_xy;
        (n.*pf)(10,20);
        cout << "SUM = " << sum(n) << "\n";

        M *op = &n;
        (op->*pf)(30,40);
        cout << "SUM = " << sum(n) << "\n";

        return 0;
 }
```

Output of Program

```
sum = 30
sum = 70
```

Local Classes

Classes can be defined and used inside a function or a block. Such classes are called local classes. Examples:

```
void test(int a)        // function
{
     .....
     .....
     class student      // local class
     {
          .....
       };
       .....
       .....
       student s1(a);          // create student object
       .....                   // use student object
}
```

Local classes can use global variables (declared above the function) and static variables declared inside the function but cannot use automatic local variables. The global variables should be used with the scope operator (::).

There are some restrictions in constructing local classes. They cannot have static data members and member functions must be defined inside the local classes. Enclosing function cannot access the private members of a local class. However, we can achieve this by declaring the enclosing function as a friend.

REFERENCES:

1.E. Balagurusamy, “Object Oriented Programming with C++”, Fourth edition, TMH, 2008.

2. LECTURE NOTES ON Object Oriented Programming Using C++ by Dr. Subasish Mohapatra, Department of Computer Science and Application College of Engineering and Technology, Bhubaneswar Biju Patnaik University of Technology, Odisha

3. K.R. Venugopal, Rajkumar, T. Ravishankar, “Mastering C++”, Tata McGraw-Hill Publishing Company Limited

4. Object Oriented Programming With C++ - PowerPoint Presentation by Alok Kumar

5. OOPs Programming Paradigm – PowerPoint Presentation